Perspectives

Prochains incréments, petits et concrets. Chacun part d'un problème mesuré (chiffres de `docs/JOURNAL.md`), propose une action, estime l'effort et fixe une mesure de succès. Les efforts sont des estimations, pas des mesures. Les gains attendus sont des hypothèses à vérifier, pas des résultats.

Protocole de mesure commun : même monde, même départ, 600 s simulées, même tracker, au moins 3 runs par configuration.

Classement gain / effort

Rang	Incrément	Gain attendu	Effort	Risque
1	P1. Plus de runs (A et B), en simulation et sur le robot réel	Élevé (conclusions solides)	0,5 jour	Faible
2	P10. Recalage LiDAR sur le robot réel (AMCL ou scan matching)	Élevé (mesure réelle plus fiable)	1 jour	Moyen
3	P2. Réduire les rotations sur place	Moyen à élevé	0,5 à 1 jour	Faible
4	P3. Valeurs demandées et valeurs appliquées (discrétisation)	Moyen (mesure plus honnête)	0,5 jour	Faible
5	P4. Diagnostiquer le gel de `map → odom`	Élevé (fiabilité, mode accéléré)	1 jour	Moyen
6	P11. Passe finale de suivi de mur (marge de 20 cm)	Moyen (couverture des bords)	1 jour	Moyen
7	P5. Coverage server de Nav2 comme référence	Moyen (comparaison)	1 jour	Moyen
8	P6. Spirale ou aléatoire amélioré pour A	Moyen	0,5 jour	Faible
9	P8. Une autre pièce	Moyen (généralité)	1 jour	Moyen
10	P7. Décomposition cellulaire de Choset	Élevé en théorie	3 jours ou plus	Élevé
—	P9. Essai sur robot réel	**Réalisé** (séances des 9 et 12/10, `docs/RESULTATS.md` §7)	—	—

P1. Plus de runs et deuxième jeu de mesures

- **Problème mesuré.** Les réglages T0 à T5 reposent sur un run par configuration. T5 contre T3 : 59.7 % contre 54.1 % à 600 s, avec des seeds différentes (5 et 1). Résultat officiel (3 runs chacun) : A 51.0 ± 6.7 %, B 44.1 ± 23.4 %. La différence des moyennes (6.9 points, A devant) est du même ordre que l'écart-type de A, et très inférieure à celui de B, tiré vers le bas par le run bloqué B3 (17.2 %). Les deux autres runs de B font 59.8 % et 55.4 %. Aucune conclusion statistique n'est possible aujourd'hui.
- **Proposition.** 5 runs de B (config T5) et 5 runs de A (mode accéléré, déjà validé), seeds fixées. Le launch de B a déjà un argument `seed` qui nomme le CSV : il suffit de l'utiliser, et de garder les rosbags. Reporter moyenne, min et max.

- **Effort.** Une demi-journée, surtout du temps machine (B en temps réel : environ 10 min par run, A accéléré : environ 3.5 min).
- **Mesure de succès.** Intervalle min-max de B publié, avec un test simple : les intervalles de A et de B se recouvrent-ils ou non à 600 s ?
- **Note.** B est déterministe en planification, mais pas en exécution (AMCL, timing de Nav2). La dispersion de B n'est pas nulle : à mesurer, pas à supposer.

P2. Réduire les rotations sur place

- **Problème mesuré.** Environ 18 % du temps de B (T5, 600 s) est passé en rotation sur place, 18 % aussi en T3. En T3 à 300 s : environ 48 s de rotation. Chaque extrémité de ligne impose un demi-tour, et un arrêt d'environ 30 s a été observé sur les petits segments entre lignes.
- **Proposition, par ordre croissant d'effort.**
    1. Orienter chaque waypoint vers le waypoint suivant (et non dans le sens de passage de sa ligne), pour que la rotation soit faite en arrivant.
    2. Régler `rotate_to_heading_min_angle` (0.785 rad aujourd'hui) : en dessous de ce seuil, le contrôleur Regulated Pure Pursuit tourne en avançant. Tester 1.2 rad et 1.57 rad.
    3. Envoyer chaque ligne comme un `Path` suivi par l'action `FollowPath` du contrôleur, au lieu d'une liste de waypoints. Le `waypoint_follower` et le `bt_navigator` sont alors court-circuités. Risque : perte de la gestion des waypoints sautés, à reprogrammer dans l'executor.
- **Effort.** 1 : 2 heures. 2 : 2 heures, une run de 300 s par valeur. 3 : une journée.
- **Mesure de succès.** Part du temps en rotation sur place sous 10 %, sans baisse du nombre de waypoints atteints (73 en T5) ni hausse des « controller patience exceeded » (2 en T5). Gain visé : couverture à 600 s supérieure à 59.7 %.

P3. Valeurs demandées et valeurs appliquées (discrétisation de la grille)

- **Problème mesuré.** La carte a des cellules de 0.05 m. Deux paramètres sont arrondis à un nombre entier de cellules :
    - `sweep_spacing` : 0.18 m demandé, 4 cellules appliquées, soit 0.20 m ;
    - `cleaning_radius` : 0.09 m demandé, 2 cellules appliquées, soit 0.10 m. Un calcul hors ligne de l'audit donne 99.8 % de couverture pour une exécution idéale du trajet, grâce à cet arrondi du rayon à 0.10 m. Il n'y a donc pas de bande non couverte en théorie. Le sujet est ailleurs : le rayon appliqué (0.10 m) est plus généreux que celui annoncé (0.09 m). La mesure est légèrement optimiste, pour A comme pour B. L'écart n'est pas chiffré sur nos runs.
- **Proposition.** Rendre les valeurs appliquées visibles et cohérentes :
    1. journaliser au démarrage les valeurs effectives (le planner le fait déjà pour l'espacement) ;
    2. choisir des paramètres multiples de la résolution (par exemple 0.20 m et 0.10 m), ou une carte plus fine ;
    3. rejouer quelques runs avec un rayon de 1 cellule (0.05 m) et de 2 cellules pour chiffrer la sensibilité de la mesure.
- **Effort.** Une demi-journée : un log, un test unitaire, quelques runs.
- **Mesure de succès.** Écart de couverture finale entre les deux rayons, pour A et B. Si l'écart est le même pour les deux stratégies, le classement n'est pas touché. Sinon, le dire.
- **Limite.** Un rayon plus petit baisse les pourcentages de A et de B. Il ne change pas la conclusion tant que l'écart reste commun : à vérifier, pas à supposer.

P4. Diagnostiquer le gel de la TF `map → odom` dans Nav2

- **Problème mesuré.** En mode accéléré, la TF `map → odom` vue par Nav2 reste figée à 89 s simulées (2 292 messages « Transform data too old »). Résultat : 8 waypoints atteints contre 69, 85 incidents contre 1, 510 m parcourus contre 61.9 m. Le mode accéléré est inutilisable pour B, ce qui interdit de diviser par 2 le temps des runs de B. Un blocage de démarrage a aussi été observé : 2 lancements sur 5, avant le correctif de l'executor. Le gel a aussi été constaté en temps réel. B3 (run officiel ; journal) : TF vue par le conteneur Nav2 figée à environ 135 s simulées, 4 329 « Transform data too old », 71 « Behavior Tree tick rate exceeded », alors que le rosbag montre AMCL publiant `map → odom` sans interruption (5 Hz, aucun trou supérieur à 1 s). T1 (essai de réglage ; journal, entrée T1/T2) : TF vue par Nav2 figée à environ 234 s simulées, 739 messages sur tout le log. Mode accéléré (vérification croisée ; journal) : figée à 89 s, 2 292 messages. Fréquence : 1 run officiel sur 3, et 3 occurrences sur environ 10 runs de B (dont 1 en accéléré).
- **Proposition.** Trier les hypothèses une par une, en gardant un run de 300 s par hypothèse :
  1. Charge CPU sous WSL : mesurer l'usage CPU de `amcl` et de `gzserver` pendant le gel.
  2. Découverte et transport DDS : essayer un autre RMW ou le profil DDS en mode local.
  3. `transform_tolerance` d'AMCL et des costmaps : valeur actuelle, puis plus grande.
  4. Pile composée contre non composée : lancer Nav2 avec `use_composition:=False` pour isoler un défaut de conteneur.
- **Effort.** Une journée.
- **Mesure de succès.** Aucun gel de la TF sur 5 runs de 600 s en temps réel. Bonus : le critère C1 de la vérification croisée (fréquence de `/cmd_vel_nav` dans ± 10 % de 10 Hz) respecté en mode accéléré, ce qui rend possible P1 en moitié de temps.
- **Limite.** L'origine exacte du gel n'est pas démontrée aujourd'hui. Ce n'est qu'une liste d'hypothèses.

P5. Coverage server de Nav2 (opennav_coverage) comme référence

- **Problème mesuré.** Notre balayage est un boustrophédon simple : 112 waypoints, contournements laissés au planificateur de Nav2, 59.7 % en 600 s sous T5. Sans autre implémentation de référence, impossible de dire si ce résultat est limité par l'algorithme ou par l'exécution.
- **Proposition.** Installer ou compiler le coverage server (`opennav_coverage`, Open Navigation) et lui fournir le même polygone de la pièce, la même largeur de passage. Exécuter son trajet sur le même robot, avec le même tracker.
- **Effort.** Une journée. Point dur : le dépôt annonce une branche et un tag pour Humble (`humble-v2`, `v1.2.1`) et un paquet apt `ros-humble-fields2cover`, mais l'installation n'a pas été testée ici. À contrôler avant de s'engager. Si l'installation échoue, abandon documenté.
- **Mesure de succès.** Courbe de couverture du coverage server à côté de A et de B, sur 3 runs.

P6. Spirale ou aléatoire amélioré pour la stratégie A

- **Problème mesuré.** A atteint 51.0 ± 6.7 % à 600 s, avec des plateaux : le robot repasse sur des zones couvertes (JOURNAL, premier résultat A). Aucune des variantes classiques de robots du commerce (spirale locale suivie de rebonds, suivi de mur) n'a été testée.
- **Proposition.** Ajouter à `random_logic.py` une variante : spirale d'Archimède au départ, puis rebonds comme aujourd'hui. Variante 2 : angle de rebond non uniforme. Garder le même tracker.
- **Effort.** Une demi-journée par variante (Python pur, tests pytest, 3 runs accélérés de 3.5 min).
- **Mesure de succès.** Moyenne sur 3 seeds supérieure à celle de A (51.0 %) de plus d'un écart-type (6.7 points). Sinon : résultat négatif documenté.

P7. Décomposition cellulaire de Choset

- **Problème mesuré.** Un pilier coupe une ligne : le robot finit le segment, puis fait un contournement par Nav2 (environ 13 s au pilier en T3). La carte compte 9 piliers. L'essai T4 (contournements calculés par le planner) a échoué : 22.4 % contre 28.5 % à 300 s, car les points intermédiaires multipliaient arrêts et rotations (38 % du temps).
- **Proposition.** Remplacer l'enchaînement de segments par une décomposition cellulaire : cellules sans obstacle, une ligne de balayage par cellule, passage de cellule à cellule par un chemin simple. Commencer par un cas à un pilier, en Python pur, avec tests sur la grille en L.
- **Effort.** Trois jours ou plus. Le plus gros incrément de la liste.
- **Mesure de succès.** Couverture finale supérieure à celle de T5, avec une distance totale inférieure (69.8 m en T5), et moins de transitions : critères à fixer avant le run.
- **Limite.** Gain non démontré. T4 a montré qu'une amélioration « évidente » peut être négative.

P8. Une autre pièce

- **Problème mesuré.** Un seul monde (`turtlebot3_world`, 5564 cellules atteignables, 13.9 m^2). Les réglages T0 à T5 sont adaptés à ce monde.
- **Proposition.** Ajouter un second monde (par exemple `turtlebot3_house`, ou une pièce à couloir en L) avec sa carte de référence, sans changer les paramètres.
- **Effort.** Une journée : cartographier (par `random_walker`, comme pour la première carte), 3 runs par stratégie.
- **Mesure de succès.** Classement A contre B inchangé, ou écart documenté. Détection des paramètres qui dépendent du monde.

P9. Essai sur robot réel : réalisé

- **Statut.** Réalisé les 9 et 12/10 sur un TurtleBot3 Burger réel, dans une zone en carton d'environ 1,6 m^2. Résultats et limites : `docs/RESULTATS.md` §7.5 à §7.7.
- **Ce qui a été obtenu.** Comparaison A contre B sur le robot (2 runs A, 2 runs B valides, 1 run B incident exclu). B est exécuté par le suiveur `coverage_follower` : Nav2 n'a pas fait avancer le robot réel (journal du 12/10). Localisation par odométrie seule (mode `static`).
- **Ce qui reste ouvert.** Peu de runs, pas de rosbag sur les runs réels, localisation sans recalage, marge d'environ 20 cm aux murs. Ces points sont repris par P1, P10 et P11.

P10. Recalage LiDAR sur le robot réel (AMCL ou scan matching)

- **Problème mesuré.** En mode `static`, `map → odom` est figée : décalage visible entre les murs de la carte et ceux du réel, dérive d'odométrie à chaque rotation sur place (A tourne environ 20 fois dans un run). Les pourcentages réels sont approximatifs, surtout pour A. Un run B (10 h 45) a sauté 2 waypoints contre un vrai carton à cause de ce décalage. AMCL avait été écarté parce que la TF `map → odom` avait 27 s de retard avec Nav2.
- **Proposition.** Une fois l'horloge du robot corrigée de façon durable, retester AMCL avec le suiveur (sans Nav2), ou un recalage par scan matching. Comparer au mode `static` sur les mêmes runs.
- **Effort.** Estimé à une journée, pas mesuré. Point dur : horloge et latence Wi-Fi.
- **Mesure de succès.** Murs de la carte superposés à ceux du LiDAR dans RViz, et âge de la TF `map → odom` sous la seconde pendant tout un run.
- **Limite.** La cause du retard de 27 s n'est pas établie ; le recalage peut échouer pour la même raison.

P11. Passe finale de suivi de mur

- **Problème mesuré.** B s'arrête à environ 20 cm des murs (marge du planificateur 15,5 cm plus tolérance d'arrivée de 8 cm). Cette bande n'est nettoyée ni par A ni par B : le pourcentage est calculé sur la même zone atteignable.
- **Proposition.** Ajouter à la fin du trajet une passe qui longe les murs à distance de sécurité réduite, avec le même tracker. Elle n'a pas été testée.
- **Effort.** Estimé à une journée, pas mesuré.
- **Mesure de succès.** Couverture de la bande de bord mesurée avant et après, sans hausse des near_collisions.
- **Limite.** Une marge plus faible augmente le risque de contact ; à tester d'abord en simulation (jumeau).

Ce qui n'est pas proposé

- Aucune promesse de « couverture complète » : en simulation (600 s), B n'a pas dépassé 60 % sur un run et 80 % n'est atteint par aucune stratégie. Sur la petite zone réelle, B atteint 80 % en 125 et 134 s (2 runs).
- Pas d'apprentissage par renforcement, hors périmètre du temps et des sources vérifiées.

P12. Filtrer les sauts de position dans la mesure

- **Constat (12/10)** : sur deux essais B bloqués, la position a sauté de plusieurs mètres quand la localisation a repris ; le `coverage_tracker` a compté ces sauts (jusqu'à 465 m « parcourus » en 600 s, alors que 0,15 m/s n'en permet que 90). Pourcentage gonflé, essais écartés.
- **Incrément** : ignorer une pose si la vitesse apparente depuis la précédente dépasse 0,5 m/s, et compter ces rejets dans `/coverage_status`.
- **Mesure de succès** : rejouer les rosbags des essais B4 et B7 (`results/bags/`) : distance plausible (90 m au plus) et couverture qui reste figée pendant le blocage.
- **Effort estimé** : 0,5 jour (estimation, non mesurée). Test pytest sur une suite de poses avec un saut.